

13 장바구니(Cart) 페이지 제작 – 함수와 변수를 아무데서나 불러서 사용하는 Redux (@reduxjs/toolkit 최신버전) props 대신 reducer로 사용

순 서	작업환경	명령어	
1	터미널에서 Redux 설치	npm install @reduxjs/toolkit react-redux	@reduxjs/toolkit 두 개의 라이브러리를 한 번에 설치 toolkit은 Redux를 더 쉽게 쓰게 해줌 (둘 다 세트로 쓰는 게 기본)
2	Redux 공식사이트 https://react-redux.js.org/ main.jsx 파일에 사용한다고 설정하기	main.jsx 상단 render(안에) 전체코드 <pre>import { BrowserRouter } from 'react-router-dom'; import { Provider } from 'react-redux'; import store from './store' // <React.StrictMode> <Provider store={store}> <BrowserRouter> <App /> </BrowserRouter> </Provider> // </React.StrictMode></pre> <pre>const root = ReactDOM.createRoot(document.getElementById("root")); root.render(// <React.StrictMode> <Provider store={store}> <BrowserRouter> <App /> </BrowserRouter> </Provider> // </React.StrictMode>);</pre> <pre>import React from "react"; import ReactDOM from "react-dom/client"; import "./index.css"; import App from "./App"; import reportWebVitals from "./reportWebVitals"; import { BrowserRouter } from "react-router-dom"; import { Provider } from "react-redux"; import store from "./store"; const root = ReactDOM.createRoot(document.getElementById("root")); root.render(// <React.StrictMode> <Provider store={store}> <BrowserRouter> <App /> </BrowserRouter> </Provider> // </React.StrictMode>); reportWebVitals();</pre>	

store.js - 리덕스 파일 만들기 (순수 자바스크립트 코드) - 앱 상태를 중앙에서 관리

순서	작업환경	명령어
3	src폴더에서 우클릭 새파일 만들기 store.js 리듀서사용해서 변수랑 함수 만들기	상단 <pre> import { configureStore, createSlice } from '@reduxjs/toolkit' // 'cart'라는 이름의 상태 만들기 (장바구니) let cart = createSlice({ name : 'cart', initialState : [{id : 1, imgurl:'fruit1.jpg', name : '수박', count : 2}, {id : 2, imgurl:'fruit2.jpg', name : '참외', count : 1}, {id : 3, imgurl:'fruit3.jpg', name : '사과', count : 1}], reducers : { // 상품 수량 1개 늘리기 addCount(state, action) { let num = state.findIndex(a => { return a.id === action.payload; }); console.log(num); console.log("내가 선택한 상품"+ action.payload); console.log("내가 추가한 상품아이디는"+ state[num].id); console.log("내가 추가한 상품갯수는"+ state[num].count); state[num].count++; }, // 상품 수량 1개 줄이기 (0개 이하는 알림 띄움) decreaseCount(state, action) { let num = state.findIndex(a => { return a.id === action.payload; }); console.log(num); if (state[num].count > 0) { state[num].count--; } else if (state[num].count === 0) { alert("상품이 더 이상 없습니다."); } } } }); </pre> 위의 코드 바로 아래

```

    },
    // 장바구니에 상품 추가하기, 이미 있으면 수량만 +1, 없으면 새로 추가
    addItem(state, action) {
      let num = state.findIndex((a) => a.id === action.payload.id);
      if (num !== -1) {
        state[num].count++;
      } else {
        state.push(action.payload);
      }
    },
    // 장바구니에서 상품 삭제하기
    deleteItem(state, action) {
      let num = state.findIndex((a) => {
        return a.id === action.payload;
      });
      state.splice(num, 1);
    },
    // 이름순으로 상품 정렬하기
    sortName(state, action) {
      state.sort((a, b) => (a.name > b.name ? 1 : -1));
    }
  }
}

// cart 함수들도 밖에서 쓸 수 있게 내보내기
export let { addCount, decreaseCount, addItem, deleteItem, sortName } = cart.actions;

```

		위의 코드 바로 아래	<pre>// 실제로 Redux에 등록해주는 부분 export default configureStore({ reducer: { cart: cart.reducer, }, }); /* createSlice: 상태랑 관련 함수들 한 번에 만들기 reducers: 상태를 바꿔주는 함수들 action.payload: 함수에 보낼 값 configureStore: 만든 상태들을 Redux에 등록하는 역할 */</pre>
--	--	----------------	---

Cart.js 장바구니 페이지 만들기			
순서	작업환경	명령어	
4	src폴더 > components > Cart.jsx 만들기	상단	<pre>import { Table } from "react-bootstrap"; import { useDispatch, useSelector } from "react-redux"; import { addCount, decreaseCount, deleteItem, sortName } from "../store.js"; import { Button } from "react-bootstrap"; import { Link } from "react-router-dom";</pre>
		상단 바로 아래	<pre>function Cart() { // 객체를 반환하므로 매번 새로운 객체가 생성되어 리렌더링이 발생할 수 있다. shallowEqual을 써서 불필요한 리렌더 방지 가능 const cart = useSelector((state) =>{ return state.cart} , shallowEqual); // dispatch는 store.js 로 요청보내주는 함수 let dispatch = useDispatch();</pre>

```

const smallProductStyle = {
  border: "1px solid #ddd",
  width: "100px",
  height: "80px",
  cursor: "pointer",
};

let textverticalAlign = {
  verticalAlign: "middle",
  textAlign: "center",
};

return (
  <>
    <div className="container">
      <div className="row">
        <div className="col-sm-12" style={{ textAlign: "center" }}>
          <h5 style={{ padding: "50px" }}>
            장바구니
          </h5>

          <Table>
            <thead>
              <tr>
                <th>id</th>
                <th>상품이미지</th>
                <th>상품명</th>
                <th>수량</th>
                <th>변경하기</th>
              </tr>
            </thead>
            <tbody>
              { /* 장바구니에 있는 상품 목록 출력 */ }
              { cart.map((( id, imgUrl, name, count ), i) => (
                <tr key={i}>
                  <td style={textverticalAlign}>{id}</td>
                  { /* 이미지 클릭 시 해당 상품 상세 페이지로 이동 */ }
                <td>

```

--	--	--

```

<Link to={`/${detail}/${id}`}>
  <img
    src={`img/${imgurl}`}
    style={smallProdcuctStyle}
  />
</Link>
</td>
{/* 상품명, 수량 보여주기 */}
<td style={textverticalAlign}>{name}</td>
<td style={textverticalAlign}>{count}</td>

<td style={textverticalAlign}>
  {/* 상품수량 + 버튼 */}
  <Button
    onClick={() => {
      dispatch(addCount(id));
    }}
    variant="outline-success"
    style={{ marginRight: "10px" }}
  >
    +
  </Button>

  {/* 상품수량 - 버튼 */}
  <Button
    onClick={() => {
      dispatch(decreaseCount(id));
    }}
    variant="outline-warning"
    style={{ marginRight: "10px" }}
  >
    -
  </Button>

  {/* 상품 삭제 버튼 */}
  <Button
    onClick={() => {
      dispatch(deleteItem(id));
    }}

```

			<pre> variant="outline-danger" > 상품삭제 </Button> </td> </tr>))) </tbody> </Table> {/* 이름순으로 정렬하는 버튼 */} <Button variant="outline-primary" onClick={() => { dispatch(sortName()); }} > 이름순정렬 </Button>{" "} </div> </div> </div> </>); } export default Cart; </pre>
Cart.jsx 전체 코드	<pre> import { Table } from "react-bootstrap"; import { useDispatch, useSelector } from "react-redux"; import { addCount, decreaseCount, deleteItem, sortName } from "../store.js"; import { Button } from "react-bootstrap"; import { Link } from "react-router-dom"; function Cart() { // 객체를 반환하므로 매번 새로운 객체가 생성되어 리렌더링이 발생할 수 있다. shallowEqual을 써서 불필요한 리랜더 방지 가능 const cart = useSelector((state) =>{ return state.cart} , shallowEqual); </pre>		

```
// dispatch는 store.js 로 요청보내주는 함수
let dispatch = useDispatch();
```

```
const smallProductStyle = {
  border: "1px solid #ddd",
  width: "100px",
  height: "80px",
  cursor: "pointer",
};
```

```
let textverticalAlign = {
  verticalAlign: "middle",
  textAlign: "center",
};
```

```
return (
  <>
    <div className="container">
      <div className="row">
        <div className="col-sm-12" style={{ textAlign: "center" }}>
          {/* 사용자 이름과 나이 보여주기 */}
          <h5 style={{ padding: "50px" }}>
            장바구니
          </h5>

          <Table>
            <thead>
              <tr>
                <th>id</th>
                <th>상품이미지</th>
                <th>상품명</th>
                <th>수량</th>
                <th>변경하기</th>
              </tr>
            </thead>
            <tbody>
              {/* 장바구니에 있는 상품 목록 출력 */}
              {cart.map(({ id, imgUrl, name, count }, i) => (
                <tr key={i}>
```



```

<td style={textverticalAlign}>{id}</td>
{/* 이미지 클릭 시 해당 상품 상세 페이지로 이동 */}
<td>
  <Link to={`/${detail}/${id}`}>
    <img src={`img/${imgurl}`} style={smallProductStyle} />
  </Link>

</td>
{/* 상품명, 수량 보여주기 */}
<td style={textverticalAlign}>{name}</td>
<td style={textverticalAlign}>{count}</td>

<td style={textverticalAlign}>
  {/* 상품수량 + 버튼 */}
  <Button onClick={() => { dispatch(addCount(id)); }} variant="outline-success" style={{ marginRight: "10px" }}> + </Button>

  {/* 상품수량 - 버튼 */}
  <Button onClick={() => { dispatch(decreaseCount(id)); }} variant="outline-warning" style={{ marginRight: "10px" }}> - </Button>

  {/* 상품 삭제 버튼 */}
  <Button onClick={() => { dispatch(deleteItem(id)); }} variant="outline-danger" > 상품삭제 </Button>
</td>
</tr>
  )))
</tbody>
</Table>

  {/* 이름순으로 정렬하는 버튼 */}
  <Button variant="outline-primary" onClick={() => { dispatch(sortName()); }} > 이름순정렬 </Button>{" "}
</div>
</div>
</div>
</>
);
}

```

export default Cart;

Detatil.jsx 상세페이지에서 주문하기와 주문상품확인하기(장바구니) 코드 추가

순서	작업환경	명령어
5	Detail.jsx	상단 import 추가 빨간색 코드 추가 <pre>import React from 'react'; import { useParams } from "react-router-dom"; import { Nav } from 'react-bootstrap' import { useState, useEffect } from 'react'; // 한 줄로 합침 import { addlItem } from './store.js' import { useDispatch } from 'react-redux' import { Button } from 'react-bootstrap' import { Link } from 'react-router-dom';</pre>
		return 위 <pre>let { paramId } = useParams();// paramId는 문자열로 오기 때문에 숫자로 바꿔줘야 한다. let [tap, setTap] = useState(0); // React Hook은 조건 없이 컴포넌트 최상단에서 호출되어야 함 let [fade2, setFade2] = useState(""); // dispatch 정의 추가 const dispatch = useDispatch(); useEffect(()=>{ setFade2('end') return ()=>{ setFade2("") } },[]) // 상품 유효성 체크 (이건 Hook 호출 이후에 실행되어야 함) let selproduct = props.fruit.find((x) => x.id === Number(paramId)); // 혹은 조건문(if) 아래에서 호출하면 안 됨 (React가 Hook 순서 기억을 못함) if (!selproduct) { return <div>해당 상품이 존재하지 않습니다.</div>;</pre>

			<pre> } const { id, imgUrl, title, content, price } = selfproduct; console.log("내가 선택한 상품은: " + id + " " + title); </pre>
		빨간색으로 코드수정 <pre> <button className="btn btn-danger">주문하기</button> </pre> 주문하기 버튼을 옆의 빨간색 코드로 대체	<pre> return (<div className={"container start " + fade2}> <div className="row" style={{ marginBottom: "50px" }}> <div className="col-md-6"> </div> <div className="col-md-6"> <h4 className="pt-5">{title}</h4> <p>{content}</p> <p>{price}</p> <Button variant="primary" onClick={() => { // dispatch(addItem({id : 1, imgurl : 'fruit1.jpg', name : 'Grey Yordan', count : 1})) dispatch(addItem({ id: id, imgurl: imgUrl.replace("img/", ""), name: title, count: 1, })); }} style={{ marginRight: "10px" }} > 주문하기 </Button> <Link to="/cart"> <Button variant="outline-success"> 주문상품 확인하기 </Button> </pre>

			<pre> </Link> </div> </div> <Nav variant="tabs" defaultActiveKey="link0"> <Nav.Item> <Nav.Link eventKey="link0" onClick={() => { setTap(0); }}>버튼0</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link eventKey="link1" onClick={() => { setTap(1); }}>버튼1</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link eventKey="link2" onClick={() => { setTap(2); }}>버튼2</Nav.Link> </Nav.Item> </Nav> <TabContent tap={tap} /> </div>); } </pre>
	Detail.jsx 전체 코드(참고)	<pre> import React from 'react'; import { useParams } from "react-router-dom"; import { Nav } from 'react-bootstrap' import { useState, useEffect } from 'react'; // 한 줄로 합침 import { addItem } from '../store.js' import { useDispatch } from 'react-redux' import { Button } from 'react-bootstrap' import { Link } from 'react-router-dom'; function Detail(props) { let { paramId } = useParams();// paramId는 문자열로 오기 때문에 숫자로 바꿔줘야 한다. let [tap, setTap] = useState(0); // React Hook은 조건 없이 컴포넌트 최상단에서 호출되어야 함 let [fade2, setFade2] = useState(""); </pre>	

```

// dispatch 정의 추가
const dispatch = useDispatch();

useEffect(()=>{
  setFade2('end')
  return ()=>{
    setFade2('')
  }
},[])

// 상품 유효성 체크 (이건 Hook 호출 이후에 실행되어야 함)

let selproduct = props.fruit.find((x) => x.id === Number(paramId));

// 혹은 조건문(if) 아래에서 호출하면 안 됨 (React가 Hook 순서 기억을 못함)
if (!selproduct) {
  return <div>해당 상품이 존재하지 않습니다.</div>;
}

const { id, imgUrl, title, content, price } = selproduct;

console.log("내가 선택한 상품은: " + id + " " + title);

return (
  <div className={`container start ` + fade2}>
    <div className="row">
      <div className="col-md-6">
        <img src={`/${ imgUrl} `} width="100%" alt={title} />
      </div>
      <div className="col-md-6">
        <h4 className="pt-5">{title}</h4>
        <p>{content}</p>
        <p>{price}</p>
        <Button
          variant="primary"

```

		<pre> onClick={() => { // dispatch(addItem({id : 2, imgUrl : 'shoes1.jpg', name : 'Grey Yordan', count : 1})) dispatch(addItem({ id: id, imgUrl: imgUrl.replace("img/", ""), name: title, count: 1, })); }} style={{ marginRight: "10px" }} > 주문하기 </Button> <Link to="/cart"> <Button variant="outline-success"> 주문상품 확인하기 </Button> </Link> </div> </div> <Nav variant="tabs" defaultActiveKey="link0" style={{marginTop:"50px"}}> <Nav.Item> <Nav.Link onClick={()=>{ setTap(0) }} eventKey="link0">버튼0</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link onClick={()=>{ setTap(1) }} eventKey="link1">버튼1</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link onClick={()=>{ setTap(2) }} eventKey="link2">버튼2</Nav.Link> </Nav.Item> </Nav> <TabContent tap={tap}/> </pre>
--	--	--

		<pre> </div>) } function TabContent({tap}){ let [fade, setFade] = useState("") useEffect(()=>{ setTimeout(()=>{ setFade('end')},10) return ()=>{ setFade("") } } ,[tap]) return (<div className={'start ' + fade}> { [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] } </div>) } } export default Detail;</pre>
--	--	--

App.jsx에 장바구니 페이지 추가하기

순서	작업환경	명령어
6	App.jsx에서	상단 import 확인
		import Cart from "../components/Cart";
		Cart 라우터 확인
		<Route path="/detail/:paramId" element={<Detail fruit={fruit} />} /> //아래 <Route path="/cart" element={<Cart/>}/>
7	브라우저에서 확인	상품 선택후 상세 페이지에서 주문하기 버튼을 누른후 주문상품 확인하기 버튼 클릭 장바구니 사이트로 가서 상품이 추가 주문되었는지 확인

브라우저에서 확인

메인화면에서 장바구니 클릭
장바구니에 이미 3개의 상품이 들어있는지 확인

과일농장 홈으로 상세페이지 장바구니 회사소개

홍길동의 장바구니

id	상품이미지	상품명	수량	변경하기
1		수박	2	+ - 상품삭제
2		참외	1	+ - 상품삭제
3		사과	1	+ - 상품삭제

이동순정렬

장바구니에 3개의 상품이 이미 주문되어 있음

메인 화면으로 돌아와서 수박상품을 클릭

메인화면에서 **햇과일 BEST**
농부가 추천하는 계절과일을 만나보세요.

상품 하나 클릭

이동순 정렬 낮은가격순 정렬 높은가격순 정렬



수박
당도선별 프리미엄 고당도 하우스수박 5~6kg
29000



참외
산지직송 성주 달콤참외 3kg
16900



사과
고씨네농장 고당도 청송사과, 햇부사 5kg
13000



바나나
델몬트 필리핀 바나나 6kg
14500



딸기
설형 딸기, 킹스베리 1박스
4,800



오렌지
캘리포니아 네이플 오렌지 4kg
12,000

수박상품에서 주문하기 버튼클릭

과일농장 홈으로 상세페이지 장바구니 회사소개



수박
당도선별 프리미엄 고당도 하우스수박 5~6kg
29000

주문하기 주문상품 확인하기

1주문버튼 클릭

버튼0 버튼1 버튼2

내용0

주문하기 버튼 클릭후 주문상품 확인하기 버튼 클릭

과일농장 홈으로 상세페이지 장바구니 회사소개



수박
당도선별 프리미엄 고당도 하우스수박 5~6kg
29000

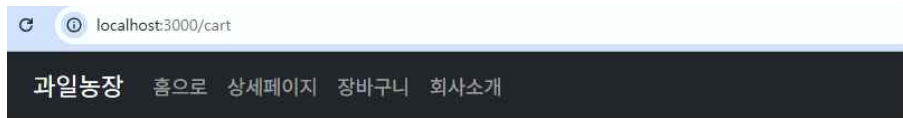
주문하기 주문상품 확인하기

2 주문상품 확인하기
버튼 클릭
(장바구니 페이지로 이동됨)

버튼0 버튼1 버튼2

내용0

수박 상품 수량이 2 + 1 = 3인지 확인



홍길동의 장바구니

장바구니에 선택한 상품이 있다면 수량만 증가

유효성체크 (내가 선택한 상품 id(1)가 있다면 action.payload 값
상품중에 같은 번호(id==1)가 있는지 체크한 다음에
수량 증감만 시킴)

id	상품이미지	상품명	수량	변경하기
2		수박	3	+ - 상품삭제
3		참외	1	+ - 상품삭제
4		사과	1	+ - 상품삭제

이름순정렬

```
let num = state.findIndex((a) => {
  return a.id === action.payload;
});
state[num].count++;
}
```

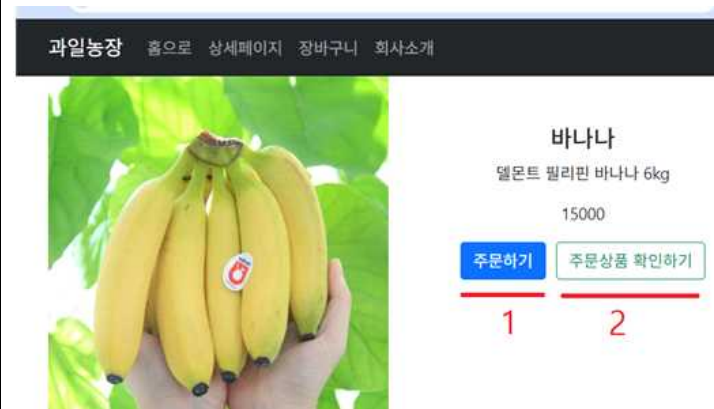
메인화면으로 다시 돌아가서 장바구니에 없는 바나나 상품 선택



메인화면에서 바나나 선택



바나나 상세페이지로 가서 1 주문하기 버튼클릭, 2 주문상품 확인하기 버튼클릭



바나나가 장바구니에 추가되었는지 확인

다른 버튼도 클릭했을 때
잘 작동하는지 확인

과일농장 홈으로 상세페이지 장바구니 회사소개

detail.js 상품상세페이지 (인자값만 줄수있다)

홍길동의 장바구니

```
<Button onClick={() => {  
  dispatch( addItem({ id: id, imgUrl: imgUrl.replace("img/", ""), name: title, count: 1, }));  
  >주문하기 </Button>
```

id	상품이미지	상품명	수량	변경하기
2		수박	3	+ - 상품삭제
3		참외	1	+ - 상품삭제
4		사과	1	+ - 상품삭제
5		바나나	1	+ - 상품삭제

이름순정렬

장바구니에 없는 상품이라면 새로 추가됨

store.js(리덕스파일) - 파라미터만 줄수있다

```
addItem(state, action) {  
  let num = state.findIndex((a) => a.id === action.payload.id);  
  if (num !== -1) {  
    state[num].count++;  
  } else {  
    state.push(action.payload);  
  }  
},
```

장바구니에 상품 추가하기, 이미 있으면 수량만 +1, 없으면 로 추가

과일농장 홈으로 상세페이지 장바구니 회사소개

홍길동의 장바구니

id	상품이미지	상품명	수량	변경하기
2		수박	4	+ - 상품삭제
3		참외	2	+ - 상품삭제
4		사과	1	+ - 상품삭제
5		바나나	1	+ - 상품삭제

다른버튼도 클릭 이름순정렬 잘되는지 확인

리듀서 문법 이해 (코드분석)

자바 스크립트 객체	자바 객체	Redux Toolkit 버전
<pre>let user = { name : '홍길동', age: 20, // 이름을 '손오공'으로 바꾸는 메서드 changeName : function() { this.name = '손오공'; }, // 나이를 전달받은 값만큼 증가시키는 메서드 increaseAge : function(value) { this.age += value; } };</pre>	<pre>public class User { String name = "홍길동"; int age = 20; void changeName() { this.name = "손오공"; } void increase(int value) { this.age += value; } }</pre>	<pre>import { configureStore, createSlice } from '@reduxjs/toolkit'; let user = createSlice({ name: 'user', // 슬라이스 이름, 라벨 initialState: { name: '홍길동', age: 20 }, // 데이터 초기값 저장(기본값) reducers: { // 안에 함수 모아두기, 데이터를 변경하는 함수들 changeName(state) { //상태를 매개변수로 받음 state.name = '손오공'; }, increase(state, action) { //변수, 함수 state.age += action.payload; } } }); // user 슬라이스에서 액션 함수들을 각각 가져오기 // 다른 파일에서 액션을 디스패치(dispatch) 할 때 사용됨 export let { changeName, increase } = user.actions; </pre>

리덕스를 사용하는 코드 쪽에서 Detail.js(상세페이지), Cart.js(장바구니)

```
// App.js
import { useDispatch, useSelector } from 'react-redux';
import store, { changeName, increase } from './store.js';

// 컴포넌트 안에서 사용
const dispatch = useDispatch();
const user = useSelector((state) => state.user);

// 버튼 클릭 시 이름 변경
<button onClick={() => dispatch(changeName())}>이름 바꾸기</button>
```

```
import { useDispatch, useSelector } from "react-redux"; //리덕스 사용
import store, {changeName, increase} from './store.js' // Provider연결, 함수사용,
```

useDispatch	Redux에 액션을 보내는 함수
useSelector	Redux 상태를 읽어오는 변수

비유

store	앱 전체 데이터(state)를 보관하는 곳(장바구니)
store	커다란 창고
state	창고 안에 들어 있는 물건들
dispatch	물건을 바꿔달라고 요청하는 것
reducer	요청을 받아서 실제로 물건을 바꿔주는 작업자

Redux Toolkit 기본 문법 정리

createSlice({ name, initialState, reducers })	상태(state)와 상태 변경 함수(reducers)를 한 번에 생성
initialState	slice의 초기 상태값 설정
reducers	상태를 바꿔주는 함수(= 리듀서) 모음
state	현재 상태를 의미, 직접 수정 가능
action.payload	액션 함수가 호출될 때 전달받는 값
export { 액션함수명 } = slice.actions	만든 액션들을 export해서 컴포넌트에서 사용 가능
configureStore({ reducer: { ... } })	여러 slice를 모아서 store 생성
slice.reducer	store 등록할 때 필요한 reducer

쇼핑몰을 Git에 배포 - 체크사항 (터미널에서 에러가 나면 안됨)

하위경로 없음	하위경로 배포	
//8253jang은 git id 자신의 id로 교체 https://8253jang.github.io/	https://8253jang.github.io/shop5/	
	package.json 마지막 라인에 추가	<pre>}, "homepage": "https://8253jang.github.io/shop5" }</pre>
	index.js basename 추가	<pre><BrowserRouter basename="/shop5"> <App /> </BrowserRouter></pre>
	App.js	이미지가 안떠도 서버에 올라가면 이미지가 경로에 맞게 나옴
	상세페이지 Detail.js	<pre></pre>
	터미널에서 확인 경로가 바뀌어도 라우터가 잘되는지	npm start 로컬호스트확인 http://localhost:5173/shop5

터미널	npm run build	작업 프로젝트 폴더 내에 dist 라는 폴더가 하나 생성됩니다. 여러분이 짰던 코드를 전부 .html .css .js 파일로 <u>변환해서 하나의 파일로</u> 담아줍니다. <u>dist 폴더 안에 안에 있는 내용을 모두 서버에 올리면 됩니다.</u> index.html이 메인페이지입니다.	
git New repository name	하위경로 없음	하위경로 배포	
	8253jang.github.io	shop5	
Repository 생성 후	dist 폴더 내의 파일을 전부 드래그 앤 드롭		
Github pages 설정	source 부분을 None이 아니라 main 저장		
배포 주소 확인	하위경로 없음	하위경로 배포	
	https://8253jang.github.io/	https://8253jang.github.io/shop5/	